



CMP6200/DIG6200

Individual Undergraduate Project (FYP)

Final Report

**SkinCare AI: A Mobile Cosmetic Skin-Concern
Analyser with Product Recommendations for
the Nepalese Market**



Student Name: Krishu Karki

Student ID: 23189624

Course: BSc (Hons) Computer Science

Supervisor: Prakash Gautam

Date: [TODO: date of submission]

Abstract

People in Nepal who want skincare advice face three barriers. Dermatologists are concentrated in a few cities. Most AI skincare tools are trained on lighter-skinned populations. Many of the products those tools suggest cannot be bought locally. This project set out to build and evaluate *SkinCare AI*, a mobile prototype that looks at a face photograph, reports visible cosmetic skin concerns and recommends products sold by Nepalese online retailers.

The artefact changed substantially during the project. The interim plan proposed a disease-and-skin-type classifier. An audit of more than 26,000 candidate images found widespread duplication, stock imagery, pre-generated augmentations and unusable skin-type labels. As a result, the scope was narrowed to five non-medical concerns: blemishes, dark spots, redness, visible pores and fine lines. Skin type is now reported by the user rather than guessed from a photo.

The final system has five parts:

- an EfficientNetB0 multi-label model trained with masked loss on 1,200+ human-reviewed and FFHQ-Wrinkle images;
- a YuNet face-quality gate;
- a catalogue of 1,500+ products scraped from Jeevee, ForEveryNG and Oriflame;
- a transparent weighted recommender;
- a FastAPI backend with a React Native (Expo) client and an optional Qwen2.5 chat assistant.

Four model versions were compared on the same duplicate-group-aware held-out set of 470+ images. The deployed model, retrained after a second round of human label review, reached a pooled label accuracy of 76.3%, a macro F1 of 0.789, a macro balanced accuracy of 0.670 and a macro ROC-AUC of 0.734, improving on the previous version on all four. Fine lines were detected reliably (ROC-AUC 0.920, balanced accuracy 0.833). Dark spots (ROC-AUC 0.580) and visible pores (0.597) remained close to chance. Class reweighting and extra melasma images were tested and rejected because they did not help.

The finished system was also tested directly. Eighteen of the 19 functional API tests passed; the chat assistant timed out under load. The face gate rejected all 300 out-of-distribution images. However, the gate also rejected 22% of held-out skin close-ups, which showed that the gate and the model do not accept the same kinds of image. Usability was assessed through a heuristic evaluation and a participant survey run through Google Forms, which combined task questions with the System Usability Scale.

The project contributes an honest, leakage-aware baseline and a working, Nepal-specific pipeline from photo to purchasable product. Its main lesson is that data quality, not model choice, limits cosmetic skin analysis at this scale.

Table of Contents

1.0 Introduction and Context	1
1.1 Problem Statement	1
2.0 Review of Existing Knowledge	2
2.1 Critical Analysis and Gap Identification	5
2.2 Project Justification	5
3.0 Project Aims, Objectives, and Scope	6
3.1 Project Aim	6
3.2 Project Objectives	6
3.3 Project Scope	7
4.0 Project Design	8
4.1 Design and Methods	8
4.2 Justification of Methods	12
5.0 Artefact Design	13
5.1 Design Specifications	13
5.2 Artefact's Structure/ Architecture / Framework	14
6.0 Implementation	17
6.1 Construction of the Artefact	18
6.2 Validation and Testing	23
7.0 Critical Evaluation	27
7.1 Impact	32
8.0 Conclusion	33
8.1 Limitations	33
8.2 Future Work	34
References	36
Appendices	39
A Project Timeline	39
B Feasibility, Risks, and Ethical Aspects	39
C Functional Test Cases	41
D Annotation Guide and Usability Study Protocol	42
E Key Code Excerpts	45

List of Figures

1	Data and model development pipeline	10
2	System architecture of SkinCare AI: Expo/React Native client, FastAPI layer, Python inference and recommendation core (TensorFlow, OpenCV, Transformers), and data assets	15
3	Screens from the running application (Expo web build, 390×844 viewport). The analysed image is an FFHQ-Wrinkle test image.	22
4	F1 overstates performance when positives dominate; ROC-AUC and balanced accuracy show where the model discriminates	24
5	Learning curve (mean of two seeds, earlier gold manifests)	25
6	Project Gantt chart	39

List of Tables

1	Comparison of reviewed work against the needs of this project	4
2	Changes from interim objectives to final objectives	7
3	Methods, tools and evidence for each objective	9
4	Design specification: functional (F) and non-functional (N) requirements	13
5	Main components of the artefact	16
6	REST API endpoints	17
7	Concern-to-ingredient and category rules used by the recommender	17
8	Technology stack	18
9	Findings of the vision data audit	18
10	Source registry decisions	19
11	Data splits of the deployed model (positive/negative known labels; unknown labels are excluded from the loss)	21
12	Held-out test performance of the deployed model (n = labelled test images per concern)	24
13	All model versions on the same held-out test set (582 known-label decisions)	25
14	Face-gate pass rate on held-out test images	26
15	Functional test summary (details in Appendix C)	27
16	Heuristic evaluation findings (author-conducted; severity 0–4)	28
17	Participant survey results (n = [TODO])	29
18	Objective attainment	30
19	Requirements against evidence	31
20	Risk assessment and mitigation (updated with outcomes)	40

1.0 Introduction and Context

Skincare is no longer a niche interest. Consumers increasingly treat it as part of everyday health, and the global beauty market has responded with personalisation platforms. These platforms promise routines matched to a person's skin, often using a selfie and an AI model (Ramrakhiani and Kalbande, 2024).

The technology behind them draws on two branches of applied machine learning:

- computer vision, where convolutional neural networks (CNNs) now rival specialists on some dermatological image tasks (Liu et al., 2020; Jones et al., 2022);
- recommender systems, which match items to user needs from item attributes and user signals (Ricci et al., 2022).

Nepal is a useful setting in which to test whether these technologies transfer beyond the markets they were built for. Its geography packs very different climates into a small area. The Terai lowlands are humid and hot. Kathmandu has marked seasonal swings and poor air quality. The high Himalayan districts combine dry cold with intense ultraviolet exposure. Each of these puts different stress on the skin (Goldust and Grant-Kels, 2024).

Access to expert advice is uneven. Qualified dermatologists are concentrated in urban centres, and people in hill and rural districts face cost, travel and time barriers to consultation (Paudel, 2020). Smartphone ownership and mobile data use, by contrast, are widespread. A self-service tool that needs only a phone camera is therefore a plausible way to make basic cosmetic skincare guidance more accessible. It would only be appropriate if it is honest about what it can and cannot see.

Global tools also create an economic problem. International skincare apps recommend brands that are often unavailable or unaffordable in Nepal. A recommendation that cannot be bought locally has little practical value. At the same time, domestic e-commerce platforms such as Jeevee and ForEveryNG now list more than 1,000 skincare products with prices, ratings and stock status. This creates an opportunity to ground recommendations in what a Nepalese user can actually purchase.

This report describes the design, construction and evaluation of *SkinCare AI*, a mobile prototype that joins these two strands. It also documents a significant change of direction. The interim report (May 2026) proposed classifying skin diseases and skin types from photographs. An audit of the available data showed that this could not be done credibly or safely (Section 6.1). The final artefact therefore detects five visible cosmetic concerns, asks the user for their skin type, and recommends locally available products. It presents its outputs as cosmetic observations, never as diagnoses.

1.1 Problem Statement

Nepalese consumers lack an accessible, trustworthy tool that turns a photograph of their face into skincare guidance they can act on. Existing AI tools are trained on data that under-represents South Asian skin (Groh et al., 2021; Daneshjou et al., 2022). They recommend

products that are largely unavailable in Nepal, and they rarely explain their outputs or state their uncertainty (Ramrakhiani and Kalbande, 2024). As a result, users either receive generic advice or rely on systems whose errors they cannot detect.

2.0 Review of Existing Knowledge

This review covers four themes that together define the design space of the project:

- deep learning for skin images;
- dataset quality and bias;
- content-based product recommendation;
- explanation and language-model assistants.

Deep learning for skin images

Liu et al. (2020) showed that a deep learning system could provide a differential diagnosis across 26 skin conditions with accuracy comparable to dermatologists. A systematic review by Jones et al. (2022) found many similar skin-cancer algorithms with high reported accuracy, but noted that most were developed on curated datasets and few had been evaluated in the populations and settings where they would be used. These systems rest on *transfer learning*: features learned on a large generic dataset such as ImageNet are reused for a smaller target domain (Zhuang et al., 2021). This is the standard approach when labelled medical images are scarce.

Tan and Le (2019) showed that scaling network depth, width and resolution together yields a family of models (EfficientNet B0–B7) that achieve high accuracy with few parameters. This makes the smaller variants attractive for resource-constrained deployments such as a mobile backend.

However, these landmark studies used tens or hundreds of thousands of clinically verified images. They also targeted *diagnosis*, a task in which each image has one answer. Cosmetic assessment is different. A face can show several concerns at once, so the problem is *multi-label*. Multi-label learning treats each label as a separate binary decision, possibly sharing a representation. It must also cope with labels that are missing rather than negative (Liu et al., 2022).

Public cosmetic datasets are small and narrowly focused. ACNE04 grades acne severity on 1,400+ images (Wu et al., 2019). FFHQ-Wrinkle provides 1,000 manually annotated wrinkle masks on faces drawn from the FFHQ collection (Moon et al., 2024; Karras et al., 2021). No public dataset labels all five concerns this project targets.

Dataset quality, leakage and bias

Two data problems are particularly relevant.

The first is *leakage*. When near-identical images appear in both the training and test sets, reported accuracy overstates true generalisation. Barz and Denzler (2020) found that 3.3% of the CIFAR-10 test images and 10% of the CIFAR-100 test images had near-duplicates in the training set. Kapoor and Narayanan (2023) catalogue leakage as a pervasive cause of irreproducible results in applied machine learning. Aggregated Kaggle datasets, which often re-upload each other's content, are especially vulnerable.

The second is *representational bias*. Groh et al. (2021) annotated the Fitzpatrick 17k dataset by skin tone and found that models performed worse on skin types under-represented in training. Daneshjou et al. (2022) confirmed on a curated, diverse clinical set that state-of-the-art dermatology models lose substantial accuracy on darker skin. A scoping review by Daneshjou et al. (2021) found that most dermatology AI datasets do not report skin tone or ethnicity at all, so bias cannot even be measured. Such disparities risk widening existing health inequities.

The SCIN dataset attempts to address representativeness. It crowdsources consented, self-taken images from the public together with self-reported skin type (Ward et al., 2024). However, its labels are clinical differentials rather than cosmetic presence or absence annotations.

Content-based and hybrid recommendation

Recommender systems are usually divided into three families (Ricci et al., 2022; Roy and Dutta, 2022):

- collaborative filtering, which learns from many users' interaction histories;
- content-based filtering, which matches item attributes to a user profile (Roy and Dutta, 2022);
- hybrids of the two.

A newly launched product has no interaction history, which is the classic "cold start" problem. In that situation content-based and rule-based methods are the only viable option. Ramrakhiani and Kalbande (2024) review AI skincare recommenders. They report that most combine a skin analysis step with ingredient-based matching, but few are evaluated rigorously and almost none are localised to a specific retail market.

Explanation and language models

Users are more willing to act on a recommendation they understand. Visual explanation methods such as Grad-CAM (Selvaraju et al., 2020) highlight the image regions that drive a CNN prediction. Natural-language explanations are an alternative, and small open language models such as Qwen2.5 (Qwen Team, 2024) can now run on commodity hardware. Language models also *hallucinate*: they state fluent claims that are unsupported by any source (Ji et al., 2023). In a product context this could mean inventing a product or an ingredient. Grounding outputs in retrieved facts and validating them in code are the standard mitigations.

Comparison of reviewed work

Table 1 places the most relevant studies side by side against the dimensions that matter for a Nepalese consumer tool. Two patterns stand out:

- studies with strong accuracy use clinical, single-label data and ignore product availability;
- studies that address cosmetic appearance cover only one concern, and none links its output to purchasable products.

Only the review by Ramrakhiani and Kalbande (2024) considers the full chain from image to product. It is a survey, however, and does not evaluate a system.

Table 1: Comparison of reviewed work against the needs of this project

Study	Task	Data	Main limitation for this project	Products
Jones et al. (2022)	Skin-cancer AI (review)	Published studies	Medical, single-label; most not tested in target populations	No
Liu et al. (2020)	Differential diagnosis	16,000+ clinical cases	Clinical photos from one US teledermatology service	No
Groh et al. (2021)	Condition classification	Fitzpatrick 17k	Shows accuracy gaps by skin type; not cosmetic	No
Wu et al. (2019)	Acne grading	ACNE04, 1,400+	One concern; severity rather than presence	No
Moon et al. (2024)	Wrinkle segmentation	FFHQ-Wrinkle, 1k masks	One concern; mostly lighter-skinned FFHQ faces	No
Ward et al. (2024)	Dataset creation	SCIN, crowd-sourced	Diagnoses, not cosmetic labels; many non-face images	No
Ramrakhiani and Kalbande (2024)	Survey of recommenders	–	Few systems rigorously evaluated; global catalogues	Yes (surveyed)
This project	5 cosmetic concerns, multi-label	2,000+ audited	Small, positive-skewed labels	Yes, Nepal

Evaluating recommenders and interactive systems

Evaluating a recommender without users is difficult. Offline metrics such as precision@k need ground-truth relevance judgements, which in skincare would require expert or user ratings (Ricci et al., 2022). Where these are unavailable, the literature relies on two weaker forms of evidence:

- transparency, meaning the user can see why an item was suggested;
- constraint satisfaction, meaning the items are in stock, in the right category and suited to the stated skin type (Roy and Dutta, 2022).

For the interface, heuristic evaluation against Nielsen’s ten heuristics remains the most common expert inspection method for mobile health apps, and it finds many problems at low cost (Galavi et al., 2024). Standardised questionnaires such as the System Usability Scale (SUS) then quantify perceived usability with real users; a mean SUS score of about 68 is widely used as the benchmark for average usability (Vlachogianni and Tselios, 2022). These two methods complement each other, and neither replaces the other.

Transparent documentation of a model’s data, metrics and limitations, as in datasheets for datasets (Gebru et al., 2021), is increasingly recommended so that downstream users can judge whether a model is fit for their population.

2.1 Critical Analysis and Gap Identification

The literature shows strong *potential* but reveals four weaknesses that matter for this project.

1. **Clinical results do not transfer to consumer cosmetic use.** Studies such as Liu et al. (2020) rely on curated clinical images and single-label diagnosis. Consumer selfies differ in lighting, pose and framing, and cosmetic concerns co-occur. There is little published evidence on multi-label cosmetic assessment evaluated with leakage controls.
2. **Public data is small, biased and often contaminated.** The available cosmetic datasets each cover one concern. Aggregated sources are prone to duplication (Barz and Denzler, 2020), and South Asian skin is under-represented (Groh et al., 2021; Daneshjou et al., 2021, 2022). Many applied papers report accuracy without describing how duplicates were controlled, which makes their results hard to trust (Kapoor and Narayanan, 2023).
3. **Recommenders are not localised.** Reviewed systems recommend from global catalogues (Ramrakhiani and Kalbande, 2024). No peer-reviewed system that we found links facial concern detection to products purchasable in Nepal.
4. **Uncertainty and explanation are rarely surfaced to users.** Deployed tools typically show a single confident label. Few expose calibrated thresholds, an “uncertain” state or a reason for each recommendation. Language-model explanations introduce their own hallucination risk (Ji et al., 2023).

The gap addressed by this project is therefore a *practical and methodological* one. There is no honest, leakage-aware baseline for multi-label cosmetic concern detection that is connected to a transparent recommender over a real Nepalese product catalogue.

2.2 Project Justification

SkinCare AI addresses each weakness directly.

- *Clinical versus cosmetic*: it replaces disease classification with five non-medical, multi-label concerns and presents results as observations with an explicit “uncertain” band.
- *Data quality*: it audits every candidate source for exact and perceptual duplicates. It keeps duplicate groups within a single data split, has humans re-label a review queue, and treats

unknown labels as missing rather than negative. Its reported metrics therefore reflect generalisation rather than memorisation.

- *Localisation*: recommendations are drawn exclusively from a catalogue of 1,500+ products scraped from Nepalese retailers, with prices in rupees and stock status.
- *Transparency*: every recommendation shows the matched ingredients and score. Any language-model output is restricted in code to products in the candidate list.

The project is relevant to Nepal's growing skincare e-commerce sector. A retailer could embed such a tool to guide customers towards suitable in-stock products, and a consumer gains a free first step before deciding whether to consult a professional. Equally important, the project documents where current public data falls short. That evidence can inform any future, locally collected dataset.

3.0 Project Aims, Objectives, and Scope

This section turns the problem into a plan. The aim and objectives below are *revised* from the interim report. Table 2 records what changed and why. Keeping the original objectives would have meant reporting against goals the evidence showed to be unsafe or unattainable.

3.1 Project Aim

The aim of this project is to design, build and evaluate a mobile prototype that detects visible cosmetic skin concerns from a face photograph. The prototype combines those concerns with a user-reported skin type and recommends explainable skincare products that can be purchased in Nepal.

3.2 Project Objectives

- O1: Dataset.** Audit every candidate image source for exact and perceptual duplicates and record its licence and provenance. Human-review at least 1,000 images against written label definitions, marking each concern as present, absent or unknown. Produce train, validation and test manifests in which no duplicate group crosses a split.
- O2: Model.** Train a multi-label EfficientNetB0 classifier for five cosmetic concerns. Select decision thresholds on validation data only. Report per-concern accuracy, F1, balanced accuracy and ROC-AUC on the held-out test set, and compare every retrained candidate against the deployed model before replacing it.
- O3: Catalogue.** Build a unified catalogue of at least 1,000 deduplicated skincare products from at least three Nepalese retailers. Each product has a price, rating, stock status, category and inferred ingredient tags.
- O4: Recommender.** Implement a transparent recommender that maps detected concerns to beneficial ingredients and categories. It ranks products by a documented weighted score, adjusts the ranking for the user's stated skin type, and returns the top five per category with the reasons for each.

O5: Application. Deliver a FastAPI backend and a React Native (Expo) mobile client that meet the requirements in Table 4, including:

- a face-quality gate;
- uncertainty display;
- a history of past checks;
- a searchable catalogue;
- an optional, grounded language-model chat assistant;
- a persistent non-medical disclaimer.

O6: Evaluation. Evaluate the artefact through held-out model metrics, documented functional test cases, an expert heuristic review, and a participant usability survey (task questions and the System Usability Scale) delivered through Google Forms. Compare the results with the interim plan and report limitations honestly.

Table 2: Changes from interim objectives to final objectives

Interim plan	Final objective	Reason for change
6 skin-type classes plus diseases from photo	5 cosmetic concerns (O1–O2); skin type from user	The skin-type folder held only 100+ images, with invalid and stock images. Disease labels (including carcinoma) implied a medical claim the data could not support.
$\geq 3,000$ images	2,000+ audited, leakage-free images	Deduplication removed thousands of repeated or augmented images. Quality was prioritised over count.
Weighted F1 $\geq 80\%$	Honest per-concern reporting	The only available test set has very few negative examples, so a single target figure would be misleading (Section 6.2).
≥ 400 SKUs from Daraz, PrettyClick, Jeevee, Bonjour Nepal	1,500+ SKUs from Jeevee, ForEveryNG, Oriflame	Scraping was feasible and permitted for these three retailers, and the product target was exceeded.
TF-IDF cosine similarity	Rule mapping plus weighted score	Ingredient lists are keyword-inferred and sparse. An explicit weighted score is easier to explain and audit.
EfficientNet-B2, PyTorch	EfficientNetB0, TensorFlow/Keras	Smaller model for CPU inference. The Keras applications API was already in use.
Web demo	Expo mobile app plus Streamlit tools	A mobile app fits the selfie use case. Streamlit was kept for internal annotation.
Usability evaluation not specified	Heuristic review plus Google Forms survey (SUS)	A short online survey lets participants test the real app and report usability in a standard, comparable form.

3.3 Project Scope

In scope:

- still photographs of a single face;
- five cosmetic concerns;
- five self-reported skin types (dry, normal, oily, combination, sensitive);
- products from three Nepalese online retailers;
- an English-language interface;
- a research prototype running on a local network.

Out of scope:

- medical diagnosis of any kind, including acne severity grading, rosacea and skin cancer;
- inferring skin type or skin tone from photographs;
- video or real-time analysis;
- collaborative filtering, because no user interaction history exists;
- Nepali-language support;
- production deployment, payment or checkout;
- collection of new images from human participants (survey participants use the app but no photos are stored).

4.0 Project Design

This section explains *how* the project was carried out: the overall approach, the method used for each objective, how the evaluation was designed, and why these methods were chosen.

4.1 Design and Methods

Research approach

The project is a **software development project with an embedded machine-learning experiment**. Its research approach is design-oriented: a working artefact is built to address a real problem, and the artefact is then evaluated to produce evidence about how well the problem is solved. Two kinds of question are asked, and each needs a different method:

- *Can the system detect cosmetic concerns reliably?* This is a quantitative question, answered with held-out model metrics.
- *Does the finished application work and is it usable?* This is answered with functional tests, an expert heuristic review and a participant survey.

The evaluation is therefore **mixed-methods**, combining numerical model results with structured usability evidence.

Development methodology

Development followed an **iterative, Agile-inspired process**. Work was organised into short iterations, each ending with a runnable version of the system. Each component (data, model,

catalogue, recommender, API, mobile client) was built and tested on its own, then integrated. Evidence from each iteration was allowed to change the plan.

The machine-learning strand followed the **CRISP-DM** cycle, the most widely used process model for data-mining projects (Schröer et al., 2021). Its six phases map directly onto this project:

1. *Business understanding*: cosmetic guidance for Nepalese users, without medical claims.
2. *Data understanding*: the audit of every candidate image source.
3. *Data preparation*: human review, label definitions and leakage-free splits.
4. *Modelling*: transfer learning with EfficientNetB0.
5. *Evaluation*: held-out metrics, with thresholds fixed on validation data.
6. *Deployment*: serving the model behind the FastAPI backend.

CRISP-DM is explicitly cyclical: an evaluation result can send the project back to data understanding. This happened twice. The first audit replaced disease classification with cosmetic concerns. Later, weak results for the concerns with few negative examples led to a second round of human label review and a retrained model (Figure 1).

Methods for each objective

Table 3 summarises the method, tools and evidence for each objective. The key method details follow the table.

Table 3: Methods, tools and evidence for each objective

Objective	Method	Main tools	Output and evidence
O1 Dataset	Data audit, human annotation, group-aware splitting	SHA-256, dHash, YuNet, Streamlit	Source registry, annotation files, validated manifests
O2 Model	Supervised transfer learning, multi-label with masked loss	TensorFlow/Keras, EfficientNetB0	Weights, evaluation.json with weight hash
O3 Catalogue	Web scraping and data cleaning	Python, pandas, rapidfuzz	Unified product CSV
O4 Recommender	Rule-based content filtering with a weighted score	Python	Ranked products with matched ingredients
O5 Application	Client-server development	FastAPI, Expo/React Native	Running app, API contract tests
O6 Evaluation	Mixed methods: metrics, functional tests, heuristic review, survey	unittest, curl, Google Forms	Evidence files, test tables, survey responses

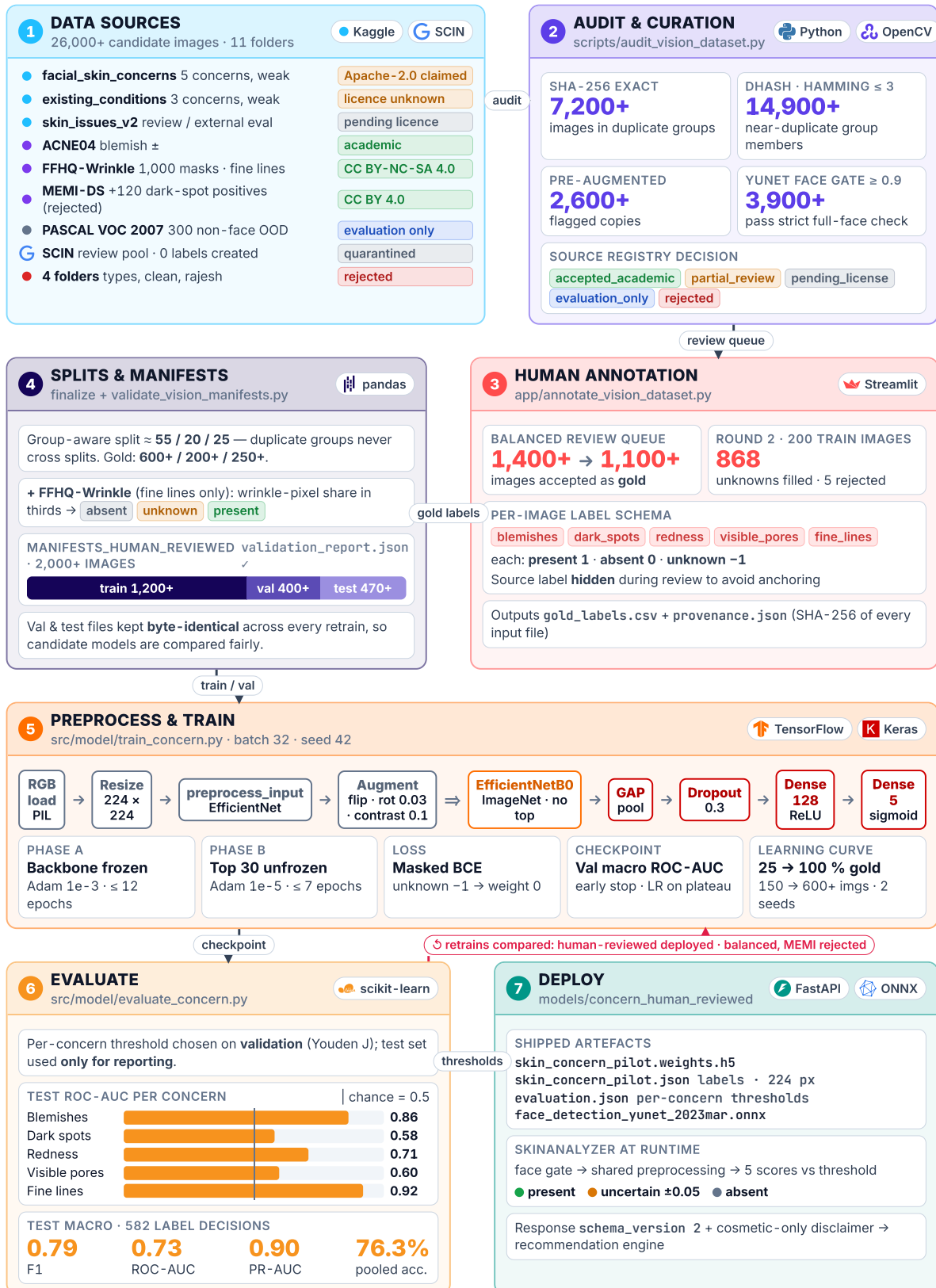


Figure 1: Data and model development pipeline

Data audit and annotation (O1). Every candidate image was hashed twice. SHA-256 found exact copies, and a 64-bit difference hash (dHash) with a Hamming radius of 3 found near-duplicates. A YuNet face detector (Wu et al., 2023) recorded whether each image showed a full face, a close-up or no face. Each source was entered in a registry with its licence, provenance and decision.

Images were then labelled in a purpose-built Streamlit tool, following a written guide (Appendix D). Each concern was marked *present*, *absent* or *unknown*, and the original folder label was hidden to reduce anchoring bias. A first queue of 1,400+ images was reviewed, followed by a second review of 200 training images whose labels were still unknown. FFHQ-Wrinkle masks (Moon et al., 2024) were converted to fine-line labels by wrinkle-pixel coverage: the lowest third became absent, the highest third present, and the middle third unknown.

Duplicate groups were assigned to a split as a unit, at roughly 55/20/25. No near-duplicate can therefore appear in both training and testing (Barz and Denzler, 2020; Kapoor and Narayanan, 2023).

Model training (O2). The model is an ImageNet-pretrained EfficientNetB0 (Tan and Le, 2019) with a five-output sigmoid head. It is trained with *masked* binary cross-entropy, so unknown labels contribute no gradient and “not annotated” is never treated as “absent” (Liu et al., 2022). Training has two phases: the head is trained with the backbone frozen (Adam, learning rate 10^{-3}), then the top 30 layers are fine-tuned (learning rate 10^{-5}). The final checkpoint is chosen on validation data only.

Each concern’s threshold maximises Youden’s J (bookmaker informedness) on validation data (Chicco et al., 2021). The test set is used only for reporting. Because positives dominate most concerns, balanced accuracy and ROC-AUC are reported next to F1, which on its own rewards always predicting “present” (Chicco and Jurman, 2020).

Catalogue and recommender (O3, O4). Product pages from Jeevee, ForEveryNG and Ori-flame were scraped to CSV and normalised to one schema. Near-duplicate products were merged using rapidfuzz token-sort similarity (≥ 85), keeping the most complete record. Category, ingredient and skin-type tags were inferred from keyword dictionaries. Each concern maps to beneficial ingredients and categories (Table 7), and products are ranked by a weighted linear score (Section 5.2).

Application (O5). A FastAPI REST backend wraps the model, face gate, recommender and language model. An Expo (React Native) client provides the interface. The two communicate only through a versioned JSON contract that is covered by a unit test.

Evaluation design (O6)

The evaluation has five strands, chosen so that each one covers a weakness of the others:

1. **Held-out model metrics** for every model version, on the same fixed test set.
2. **A learning-curve study**, retraining on 25–100% of the data, to test whether more data would help.
3. **Black-box functional tests**: 19 documented cases run against the live API.

4. **A heuristic evaluation** of the running app against Nielsen's ten heuristics (Galavi et al., 2024).
5. **A participant usability survey.** Participants install and use the app, complete four set tasks (check a photo, read the result, find a product, ask the assistant), then answer a Google Form. The form contains consent, anonymous background questions, task completion and ease ratings, the ten-item System Usability Scale (Vlachogianni and Tselios, 2022), and questions about trust and understanding of the results. No names, emails or photos are collected. The full protocol is in Appendix D.

4.2 Justification of Methods

Iterative process and CRISP-DM rather than a fixed plan. The data's quality was unknown at the start. A fixed, waterfall-style plan would have committed the project to the disease classifier before the audit showed it was unsafe. An iterative process made it possible to change direction on evidence (Schröer et al., 2021).

Transfer learning rather than training from scratch. With only 1,200+ training images, a CNN trained from scratch would overfit badly. ImageNet features give a strong starting point for texture and colour (Zhuang et al., 2021). EfficientNetB0 was chosen over the planned B2 because it has about 5.3 million parameters, compared with about 9.2 million for B2. On a CPU-only backend, the smaller model's speed outweighed any small accuracy gain that B2 might have given.

Multi-label with masking rather than multi-class. Concerns co-occur on real faces, so a single softmax label would force the model to pick one. Masking lets partially labelled images still contribute: an image labelled only for fine lines still trains the fine-lines output.

Audit and human review rather than accepting folder labels. The folder labels in the Kaggle sources were contaminated (Section 6.1). Training on them would have produced impressive but meaningless accuracy. Human review is slower, but it is the only way to obtain labels whose meaning is defined.

Skin type from the user rather than from the photo. The only skin-type image source was rejected (100+ images, including stock photographs). Oiliness and dryness are also partly tactile, so they are hard to judge from one photo. Asking the user is more reliable and more respectful.

Weighted rules rather than TF-IDF similarity or collaborative filtering. Collaborative filtering needs interaction histories that do not exist (Ricci et al., 2022). TF-IDF over ingredient text was planned, but scraped listings rarely include full ingredient lists: 65% of products have no inferable ingredient at all. A weighted score over explicit features is simpler, degrades predictably and can be explained line by line (Roy and Dutta, 2022).

Grounded language model rather than free generation. Qwen2.5 (Qwen Team, 2024) runs locally without a paid API. Its outputs are restricted in code, not only by the prompt, to product IDs in the candidate set. It is optional, and the core flow never depends on it. This limits the hallucination risk (Ji et al., 2023).

Heuristic review and an online survey together. A heuristic review finds many interface problems cheaply, but it reflects only the evaluator’s view (Galavi et al., 2024). A survey with SUS captures how real users perceive the app and gives a score that can be compared with published benchmarks (Vlachogianni and Tselios, 2022). Google Forms was chosen because it is free, needs no account for participants, records responses anonymously, and exports directly to CSV for analysis.

5.0 Artefact Design

This section gives a factual description of the artefact as built: its requirements (Table 4) and its architecture (Figure 2).

5.1 Design Specifications

Table 4: Design specification: functional (F) and non-functional (N) requirements

ID	Requirement	Obj.
F1	Accept a JPEG/PNG face photo up to 10 MB from camera or gallery	O5
F2	Reject images without a detectable face and explain how to retake the photo	O5
F3	Return a score, threshold and status (<i>present</i> , <i>absent</i> or <i>uncertain</i>) for each of five concerns	O2
F4	Accept an optional user skin type (dry, normal, oily, combination, sensitive); never infer it from the photo	O4
F5	Recommend up to five products per relevant category, with the matched ingredients and a score	O4
F6	Suggest a morning and evening routine derived from the detected concerns	O4
F7	Browse, search, filter and sort the catalogue; view product detail; save favourites	O5
F8	Keep a local history of past checks	O5
F9	Answer free-text questions through a grounded chat assistant, falling back to rules if the language model is unavailable	O5
F10	Show a non-medical disclaimer on every result and chat screen	O5
N1	No medical labels; wording must describe observations, not diagnoses	–
N2	Thresholds are set on validation data only; the test set is used only for reporting	O2
N3	Photos are not stored on the server; history is kept on the device only	–
N4	Warm analysis completes in under 2 seconds on a laptop CPU	O5
N5	Runs entirely on free, open-source software without paid APIs	–
N6	Structured API errors (400/404/422/500) so the client can show helpful messages	O5

5.2 Artefact's Structure/ Architecture / Framework

Architectural style

SkinCare AI uses a **three-tier client-server architecture** (Figure 2):

1. a *presentation tier*, the Expo/React Native mobile client;
2. an *application tier*, a stateless FastAPI REST service that holds the model, face gate, recommender and language model in memory;
3. a *data tier* of read-only files: model weights and thresholds, the product catalogue and the ingredient mappings.

Three design principles shape this structure:

- **Separation of concerns.** The client only collects input and displays results. All analysis and ranking happens on the server, so the model can be replaced without changing the app.
- **Statelessness and privacy by design.** The server stores no photos and no user data. Each request carries everything needed to answer it, and history and favourites are kept only on the device (requirement N3).
- **Graceful degradation.** The language model is optional. If it is unavailable, slow or returns invalid output, a deterministic rule-based answer is used, so the core photo-to-product flow never depends on it.

Components

Table 5 lists the main components, what each is responsible for and where it lives in the code.

API interface

The client and server communicate only through the REST endpoints in Table 6. Analysis responses carry a schema version, the model version and the disclaimer, and their structure is fixed by a contract unit test.

Analysis request flow

When the client uploads a photo to `/api/analyze`, the server processes it in five steps:

1. It checks the MIME type and size of the upload.
2. It decodes the image to RGB and runs YuNet on a copy resized to at most 320 px. If no face scores at least 0.9, it returns a *rejected* result that explains how to retake the photo.
3. It preprocesses the image exactly as during training: bilinear resize to 224×224 followed by EfficientNet normalisation. This code is shared by training, evaluation and serving.
4. It predicts five sigmoid scores. A score within ± 0.05 of its concern's threshold is labelled *uncertain*, a higher score *present*, and a lower score *absent*.

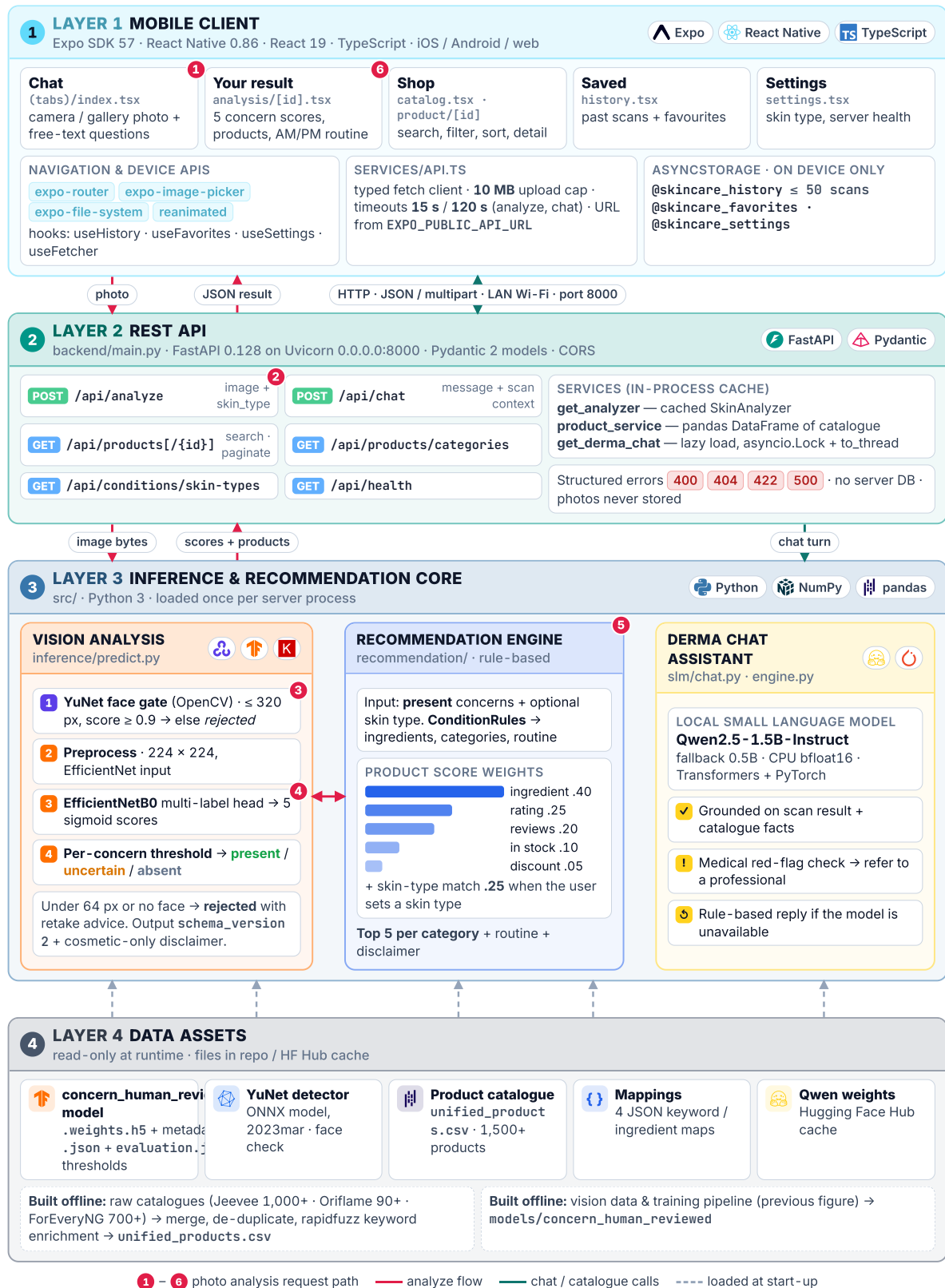


Figure 2: System architecture of SkinCare AI: Expo/React Native client, FastAPI layer, Python inference and recommendation core (TensorFlow, OpenCV, Transformers), and data assets

Table 5: Main components of the artefact

Component	Responsibility	Implementation
Mobile client	Tabs for Chat, Shop, Saved and Settings; photo capture; result, routine and product screens; on-device history and favourites	mobile/app, mobile/services/api.ts, AsyncStorage hooks
API layer	Validates uploads and parameters, routes requests, returns structured errors (400/404/422/500)	FastAPI routers in backend/routes
Face gate	Rejects photos without a clear face (YuNet score ≥ 0.9)	SkinAnalyzer with OpenCV FaceDetectorYN
Concern model	Shared preprocessing, five sigmoid scores, per-concern thresholds and status	SkinAnalyzer in predict.py; shared concern_preprocessing.py
Recommender	Maps present concerns to ingredients and categories; ranks products by weighted score	RecommendationEngine, ConditionRules, scoring.py
Language-model layer	Optional re-ranking and grounded chat; rule-based fallback; red-flag signposting	SlmEngine, DermaChat, SlmRecommender (Qwen2.5)
Data assets	Model weights, evaluation.json thresholds, product catalogue, ingredient mappings	models/, data/enriched, data/mappings

5. It passes the concerns marked *present*, and the stated skin type, to the recommender, and returns the combined result.

Core modules

Model. The network is EfficientNetB0 without its ImageNet head, followed by global average pooling, dropout (0.3), a 128-unit ReLU layer and a 5-unit sigmoid output. It has about 4.2 million parameters in total. Per-concern thresholds are loaded from the model's `evaluation.json` file, so re-evaluating the model updates the live app without any code change.

Recommender. Table 7 lists the rule map. For each category relevant to the present concerns, every product in that category receives a score:

$$s = w_i I + w_r \frac{\text{rating}}{5} + w_v \frac{\log(1+\text{reviews})}{\log 1001} + w_a \mathbf{1}[\text{in stock}] + w_d \min(2 \cdot \text{discount}, 1) + w_t T \quad (1)$$

Here I is the fraction of target ingredients that the product contains, and T is its match to the stated skin type. The weights are $(w_i, w_r, w_v, w_a, w_d, w_t) = (0.40, 0.25, 0.20, 0.10, 0.05, 0)$ when no skin type is given, and $(0.35, 0.20, 0.12, 0.06, 0.02, 0.25)$ when one is. The top five products per category are returned together with the ingredients they matched. If no concern is present, a basic cleanser, moisturiser and sunscreen routine is suggested.

Language-model layer. Qwen2.5–1.5B–Instruct, with a 0.5B fallback, runs on the CPU. It is used in two places:

- optionally, to reorder and justify the top candidates in an analysis result;

Table 6: REST API endpoints

Method	Path	Purpose
GET	/api/health	Connection check used by the app's "Test connection" button
POST	/api/analyze	Multipart photo plus optional skin type; returns concerns, routine and recommendations
POST	/api/chat	Question plus optional latest result; returns a grounded reply and product cards
GET	/api/products	Catalogue search, filter by category and skin type, sort and paginate
GET	/api/products/categories	List of product categories
GET	/api/products/{id}	Product detail, or 404
GET	/api/conditions/skin-types	The five skin types with recommended ingredients

Table 7: Concern-to-ingredient and category rules used by the recommender

Concern	Target ingredients	Categories
Blemishes	salicylic acid, niacinamide, tea tree, zinc, glycolic acid	cleanser, serum, moisturiser, spot treatment, toner
Dark spots	vitamin C, niacinamide, kojic acid, arbutin, glycolic acid, azelaic acid	cleanser, serum, moisturiser, spot treatment, sunscreen
Redness	centella asiatica, ceramides, aloe vera, niacinamide, panthenol	cleanser, moisturiser, sunscreen, serum
Visible pores	niacinamide, salicylic acid, zinc, glycolic acid	cleanser, toner, serum, moisturiser
Fine lines	retinol, hyaluronic acid, peptides, ceramides, vitamin C	cleanser, serum, moisturiser, sunscreen

- for the chat assistant.

The prompt contains only catalogue facts for the candidate products. The response is parsed as JSON, and any product ID not in the candidate list is discarded in code. If the model is unavailable, or produces nothing valid, a deterministic rule-based answer is returned instead.

Technology stack. Table 8 summarises the stack.

6.0 Implementation

This section describes how the artefact was built (Section 6.1) and how it was validated (Section 6.2). It links each step to the problem statement: producing trustworthy, locally actionable skincare guidance.

Table 8: Technology stack

Layer	Technologies
Model & data	Python 3.9, TensorFlow 2.16 / Keras, OpenCV (YuNet), pandas, NumPy, rapidfuzz, Pillow
Backend	FastAPI, Uvicorn, Pydantic, Hugging Face Transformers (Qwen2.5)
Mobile	Expo SDK, React Native, expo-router, TypeScript, AsyncStorage, expo-image-picker
Tooling	Streamlit (annotation and demonstration), Git, unittest

6.1 Construction of the Artefact

Iteration 1: disease and skin-type prototype (August 2026)

The first version followed the interim plan. Images from several Kaggle collections were merged into folders for seven conditions (including carcinoma) and three skin types. A multi-task EfficientNetB0 was trained on these folders. The Streamlit and mobile interfaces showed the predicted condition, and a carcinoma prediction triggered a “see a dermatologist” message.

This iteration proved that the full pipeline could be integrated, from photo to API to recommendations. However, as the progress review recorded, it produced no trustworthy metrics. Manual trials also suggested poor real-world accuracy.

Iteration 2: data audit (September 2026)

Rather than tune the model further, the data was audited. This proved to be the most important decision in the project. Table 9 summarises the findings across more than 26,000 files drawn from the original folders and four additional candidate collections.

Table 9: Findings of the vision data audit

Finding	Count
Files audited	26,000+
Pre-generated (already augmented) images	2,600+
Exact-duplicate groups / images involved	3,200+ / 7,200+
Perceptual-duplicate groups / images involved	5,200+ / 14,900+
Perceptual groups spanning different sources	2,400+
Images passing the strict full-face gate	3,900+

More than half of all images belonged to a perceptual-duplicate group. Thousands of duplicates crossed source boundaries, which shows that the Kaggle collections had re-uploaded each other. The “cleaned” train/validation split that came with the original data contained repeated stock images and augmentations, so it leaked into its own validation set. Any accuracy measured on it would have been inflated (Barz and Denzler, 2020).

The skin-type folder held only 100+ images, including stock photographs whose labels could not be verified. The condition folders mixed medical diagnoses with cosmetic appearance and had no provenance.

These findings led to three decisions:

1. drop disease labels, and with them any implied medical claim;
2. stop predicting skin type from photographs and ask the user instead;
3. redefine the task as five visible cosmetic concerns, labelled by a human against written definitions.

The source registry (Table 10) records the outcome for every source.

Table 10: Source registry decisions

Source	Images	Decision	Reason
Original condition folders	7,600+	Partial	Only acne, dark-spot and rosacea folders reviewed; no provenance
Kaggle facial skin concerns	4,200+	Partial	Useful coverage; watermarks and label noise
Kaggle skin issues v2	9,700+	Pending	Unknown licence; heavy overlap
Original “cleaned” split	3,200+	Rejected	Stock images, augmentations, leakage
Kaggle normal/oily/dry	1,400+	Rejected	Stock images, invalid skin-type labels
Original skin-type folder	100+	Rejected	Too small; photo-based skin type out of scope
ACNE04 (Wu et al., 2019)	390+	Accepted	Academic dataset, known provenance
FFHQ-Wrinkle (Moon et al., 2024)	1,000	Accepted	Manual wrinkle masks, CC BY-NC-SA
SCIN (Ward et al., 2024)	70	Quarantined	0 of 70 passed the full-face gate; no cosmetic labels
PASCAL VOC 2007 (Everingham et al., 2010)	300	Test only	Non-face images for face-gate testing

Iteration 3: annotation, training and deployment

A review queue of 1,400+ images, balanced by source and concern, was labelled in a custom Streamlit annotator. Of these, 1,100+ images were accepted and the rest were rejected as unusable. Most labels remained *unknown*, which is the honest answer for close-ups showing one region.

Negative examples were very scarce. In the first gold set, fewer than 20 images were labelled as having no blemishes, and fewer than 35 as having no redness. Fine lines had the most balanced labels, so FFHQ-Wrinkle was added to strengthen that concern.

The first manifests contained 2,000+ images. Training used seed 42, horizontal-flip, rotation and contrast augmentation, and the two-phase schedule described in Section 4. Training refuses to overwrite existing weights, so every run is preserved. This model (`concern_v2`) was deployed first, and its weak results for blemishes, redness and pores (Section 6.2) were traced to the very small number of negative training labels: only 9 for blemishes and 19 each for redness and pores.

Two further engineering problems were found and fixed during this iteration.

- **Train/serve preprocessing mismatch.** The backend originally resized images with a different method from the one used in training. Moving preprocessing into one shared module raised the pooled label accuracy of `concern_v2` on the test set from 70.3% to 73.2% with no retraining. This shows how silent inconsistencies can erode deployed performance.
- **Degenerate thresholds.** Choosing thresholds by maximising F1 on validation data set the dark-spot and pore thresholds to 0.0, so those concerns were *always* reported as present. Switching to Youden's J , which rewards specificity as well as recall, produced meaningful thresholds (Table 12).

Iteration 4: second label review and retraining

Two attempts were made to fix the shortage of negative examples.

- **Class reweighting** gave each concern's positive and negative labels equal total weight. It did not help on the test set and was rejected (Table 13).
- **A second human review** targeted the 200 training images whose dark-spot, redness or pore labels were still unknown. Every image was re-examined against the same guide. The merge filled 868 previously unknown labels and removed five images judged unusable. Seven disagreements with existing fine-line labels were logged and the original labels were kept. Validation and test files were copied unchanged and verified by hash.

The second review raised the number of negative training labels sharply: from 9 to 159 for blemishes, from 19 to 188 for redness, and from 19 to 71 for visible pores (Table 11). The model was then fine-tuned from the `concern_v2` weights with the same seed and no class reweighting. The best checkpoint on validation macro ROC-AUC was selected (0.706, compared with 0.686 for `concern_v2`). Its thresholds were again chosen on validation data only. This model, `concern_human_reviewed`, scored higher than `concern_v2` on all four macro test metrics and is now the deployed model.

A final experiment added 120+ melasma photographs from the MEMI dataset to the training split as extra dark-spot positives. It did not improve dark spots and lowered overall accuracy, so it was not deployed.

Table 11: Data splits of the deployed model (positive/negative known labels; unknown labels are excluded from the loss)

Split	Images	Blemishes	Dark spots	Redness	Pores	Fine lines
Train	1,200+	274/159	204/188	249/188	228/71	362/390
Validation	400+	68/4	36/22	64/6	33/11	123/95
Test	470+	97/4	65/22	97/8	46/9	116/118

Catalogue and recommender

Scraping produced 700+ ForEveryNG, 1,000+ Jeevee and 90+ Oriflame rows. After normalisation and fuzzy deduplication, 1,500+ unique products remained: 800+ from Jeevee, 600+ from ForEveryNG and 70+ from Oriflame. They cover 650+ brands, with a median price of Rs. 799, and 99% of them are in stock.

Keyword inference assigned a category to 82% of products. However, it found at least one target ingredient for only 35% of products (500+), and a skin-type tag for 38%, because most listings name only headline actives. This limits the ingredient component of the score (Section 7).

Mobile application

The Expo client has four tabs: Chat, Shop, Saved and Settings. Photos are added from the chat screen (the “+” button), which keeps analysis and follow-up questions in one conversation. Results open a detail screen with the following elements:

- score bars with a threshold tick;
- the status of each concern;
- a morning and evening routine;
- recommendations grouped by category, with “Fits” ingredient tags.

The server address is discovered automatically on the local network, and structured API errors are translated into plain-language messages. Figure 3 shows screenshots captured from the running application.

User journey. A first-time user opens the app on the Chat tab, where an empty-state prompt invites them to tap “+” to check a photo or to type a question. Before any analysis, the user can set a skin type in Settings; the screen states that skin type is “never guessed from photos”, and it may be left unset. Tapping “+” offers the camera or the gallery, with an instruction to use “one clear, front-facing photo of your whole face in daylight”. The result appears as a card that opens the detail screen. Tapping a product opens its detail page, where it can be saved to the user’s “shelf”. Every result is stored in the on-device history under the Saved tab, and the assistant receives the latest result as context, so follow-up questions can refer to the user’s own concerns and recommended products.

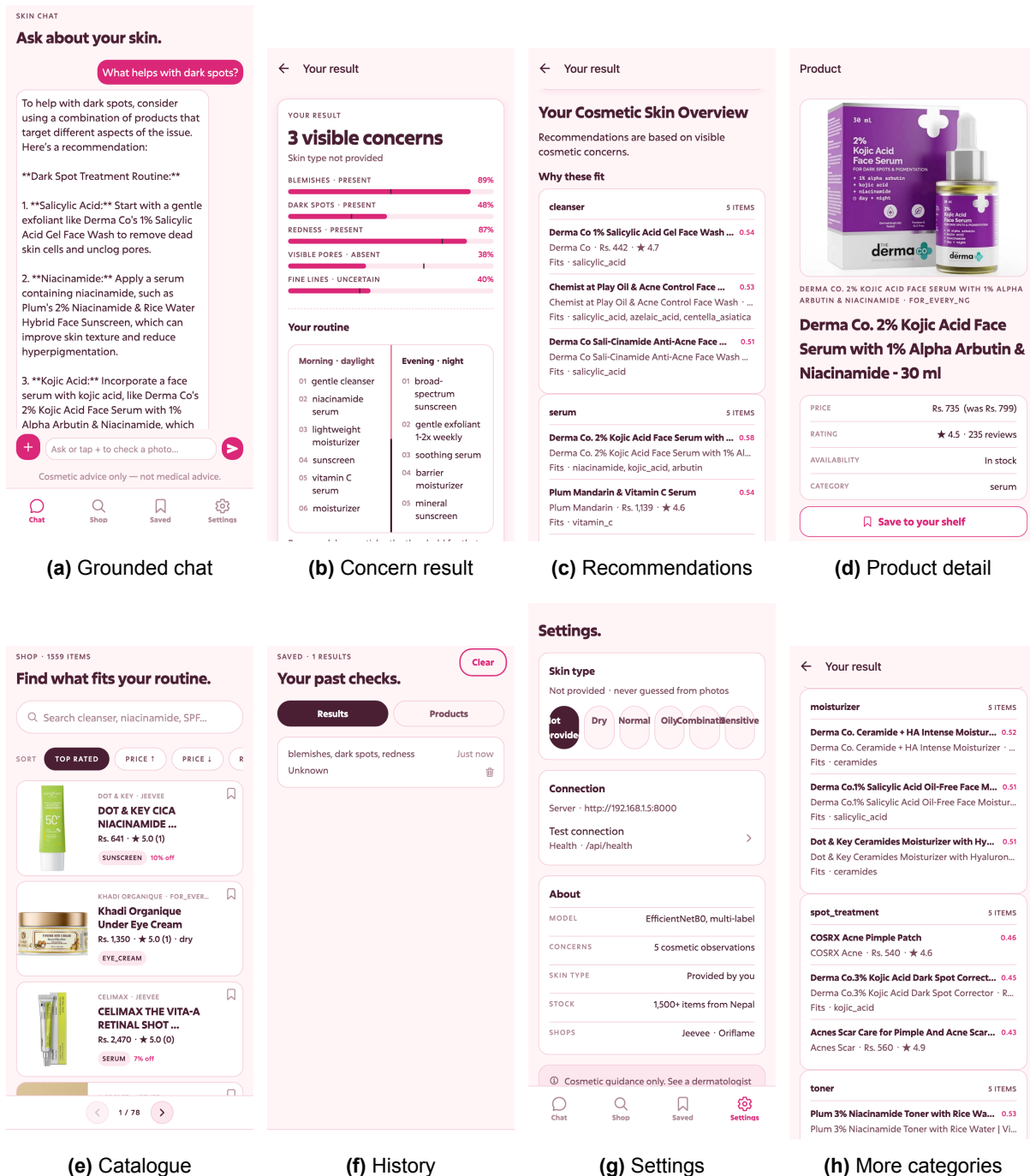


Figure 3: Screens from the running application (Expo web build, 390×844 viewport). The analysed image is an FFHQ-Wrinkle test image.

Key development challenges

Several practical problems consumed more time than planned. Each shaped the final design.

- **Native photo upload.** On physical devices, photos selected in React Native could not be sent reliably as form data using the web-style `fetch` pattern. Uploads were rewritten to use `expo-file-system`. A 10 MB size check was added on the client as well as the server, so that users receive an immediate, specific error rather than a generic failure.
- **Local networking.** The phone and the laptop running the backend must share a network, and the backend's address changes between networks. The client derives the server address from the Expo development host automatically. A "Test connection" diagnostic calls `/api/health` and reports exactly which URL failed. This turned a frequent source of confusion into a one-tap check.
- **Language-model integration.** Running Qwen2.5 inside the API process blocked other requests. Generation was moved to a worker thread behind an asynchronous lock. The analysis flow was made independent of the language model, and a rule-based fallback now answers when the model is slow, unavailable or returns invalid JSON.
- **Reproducibility.** After the preprocessing mismatch was discovered, a SHA-256 hash of the weights, the manifest paths and the preprocessing description were written into every evaluation file. This allows any reported number to be traced to the exact model and data that produced it.

These challenges map directly to the problem statement. Trustworthy guidance requires not only an accurate model, but also an application that fails clearly, explains its errors and can be reproduced.

6.2 Validation and Testing

Validation used five complementary kinds of evidence. All raw outputs are stored in `report/final/evidence/`.

Unit tests

Nine test modules in `scripts/` were run. All passed, including a pipeline integration test that runs the real model and recommender end to end. Together they cover:

- the API response contract;
- audit hashing;
- manifest and training logic (mask handling, balancing and non-overwrite);
- the SCIN quarantine;
- the language-model guardrail, which verifiably drops a fabricated product ID.

The suite is small, and only some of its checks are formal assertions.

Model performance on held-out data

Table 12 reports test metrics for the deployed `concern_human_reviewed` model. Thresholds were chosen on validation data. Figure 4 compares F1, ROC-AUC and balanced accuracy, and Table 13 compares all four model versions on the same test set.

Table 12: Held-out test performance of the deployed model (n = labelled test images per concern)

Concern	n (+/-)	Thr.	Acc.	Prec.	Recall	F1	Bal. acc.	ROC-AUC
Blemishes	101 (97/4)	0.50	0.901	0.978	0.918	0.947	0.709	0.863
Dark spots	87 (65/22)	0.43	0.506	0.775	0.477	0.590	0.534	0.580
Redness	105 (97/8)	0.75	0.771	0.951	0.794	0.865	0.647	0.709
Visible pores	55 (46/9)	0.81	0.600	0.900	0.587	0.711	0.627	0.597
Fine lines	234 (116/118)	0.29	0.833	0.824	0.845	0.834	0.833	0.920
Macro	582 decisions	–	0.722	0.885	0.724	0.789	0.670	0.734

Pooled accuracy over all 582 known-label decisions: **76.3%**. Macro PR-AUC: 0.904.

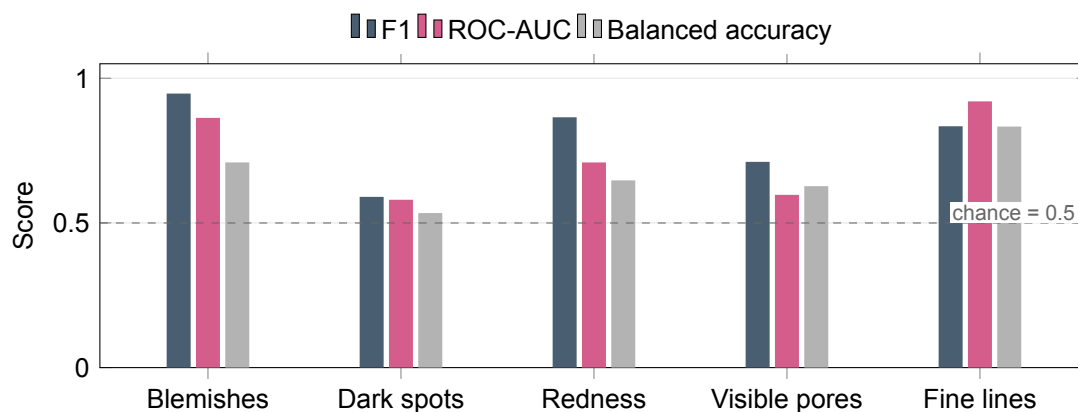


Figure 4: F1 overstates performance when positives dominate; ROC-AUC and balanced accuracy show where the model discriminates

Three results stand out.

- **Fine lines are learned well.** On the only balanced concern, the model reaches an ROC-AUC of 0.920 and a balanced accuracy of 0.833, up from 0.911 and 0.795.
- **More negative labels helped where they were added.** Redness accuracy rose from 55.2% to 77.1%, although its balanced accuracy barely changed (0.643 to 0.647). Blemish balanced accuracy rose from 0.610 to 0.709, and pores from 0.560 to 0.627. Even so, the test set still holds only 4 negative blemish images and 8 negative redness images, so these figures move by large steps when a single image changes. The high F1 scores for blemishes and redness still flatter the model: with 97 positives and 4 negatives, always saying “present” would give $F1 \approx 0.98$.
- **Dark spots did not improve.** Dark-spot accuracy fell from 64.4% to 50.6% and ROC-AUC stayed at about 0.58. The deployed model is the best overall, but it is worse than `concern_v2` on this one concern, and dark-spot output should be treated as unreliable.

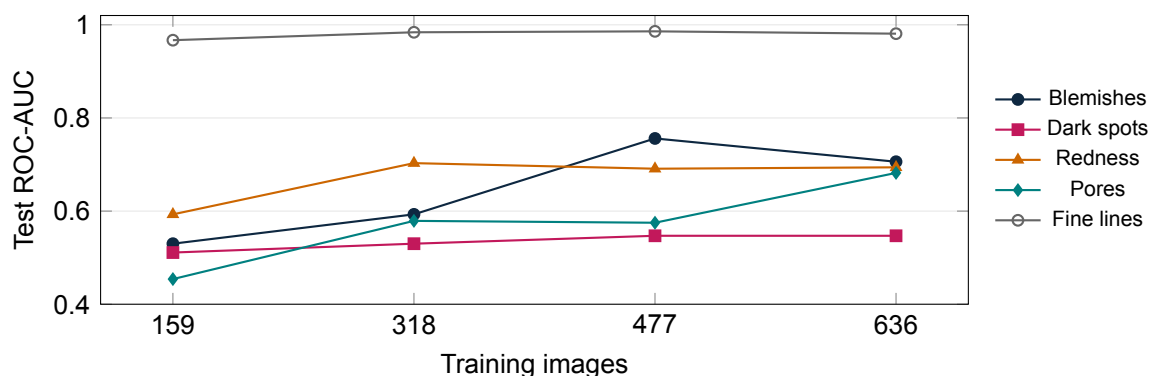
Table 13: All model versions on the same held-out test set (582 known-label decisions)

Model	Pooled acc.	Macro F1	Bal. acc.	ROC-AUC	Decision
concern_v2 (first labels)	73.2%	0.779	0.635	0.717	Replaced
Class-balanced candidate	74.6%	0.736	0.617	0.708	Rejected: redness specificity 0
Human-reviewed (deployed)	76.3%	0.789	0.670	0.734	Deployed
Human-reviewed + MEMI melasma	69.8%	0.701	0.662	0.750	Rejected: accuracy fell

Run-to-run variation. Training on the Apple GPU (tensorflow-metal) was not fully reproducible: retraining the same recipe with the same seed gave 72.2% and 78.9% macro accuracy. Over three runs, the human-reviewed recipe averaged 74.4% macro accuracy and 0.746 ROC-AUC. Differences of a few points between single runs are therefore within noise. The deployed weights are the stored run whose metrics are reported above, identified by the SHA-256 hash in its evaluation file.

Learning curve. To test whether more data would help, the model was retrained on 25, 50, 75 and 100% of the earlier gold training set (150 to 600+ images), with two seeds each (Figure 5). ROC-AUC rose with data for blemishes (0.53 to 0.71) and pores (0.45 to 0.68). Dark spots stayed near 0.55. Fine lines were already at about 0.97 on that smaller, easier test set.

These trends suggest that more labelled data would help most concerns, but that dark spots may also need a better label definition. The run-to-run standard deviation (up to 0.20 for blemishes) is large relative to the gains, so these trends are indicative only.


Figure 5: Learning curve (mean of two seeds, earlier gold manifests)

Negative experiments. Two candidates were rejected (Table 13). Class reweighting slightly improved validation ROC-AUC (0.686 to 0.698), but on the test set macro F1 fell to 0.736 and redness specificity dropped to 0, with all eight negatives misclassified. Adding MEMI melasma photos raised macro ROC-AUC to 0.750 but lowered pooled accuracy to 69.8% and left dark-

spot ROC-AUC near chance (0.57). Both results point the same way: reweighting or adding positives cannot create the variety of *negative* examples that is missing from the data.

Face-gate validation

The YuNet gate (threshold 0.9) rejected all 300 PASCAL VOC non-face images. It was then run on every held-out test image to check how it would treat real inputs (Table 14).

Table 14: Face-gate pass rate on held-out test images

Source / framing	Images	Passed	Rate
FFHQ-Wrinkle, full face	211	211	100%
Kaggle concerns, full face	40	34	85%
Original conditions, full face	23	19	83%
ACNE04, full face	15	11	73%
Kaggle concerns, close-up	106	64	60%
Original conditions, close-up	54	14	26%
ACNE04, close-up	30	20	67%
All	479	373	77.9%

Of the held-out images, 22% would be rejected by the deployed app before reaching the model. Most were facial close-ups, the framing that carries most of the blemish, pore and dark-spot labels. The model metrics therefore describe a wider input range than the app accepts. Conversely, the app never sees many of the close-ups on which the concern model was trained. This mismatch between the gate and the model is discussed in Section 7.

Functional testing

Nineteen black-box test cases were executed against the running API with `curl`. They were first run on 30 September 2026 and re-run on 1 October 2026 after the human-reviewed model was deployed. Appendix C lists them in full, and Table 15 summarises the re-run. Eighteen passed. One failed: TC15, the chat question. In the first run it returned a grounded reply after 55 s. In the re-run it gave no response within 240 s, and a retry also timed out after 500 s. The cause is a design weakness: chat generations run one at a time behind a lock with no timeout, so a request that the client has abandoned keeps running and blocks every later one. Warm analysis latency over 10 runs had a median of 0.18 s (maximum 0.24 s) on an Apple M4 CPU, which satisfies N4.

Usability: heuristic evaluation

The running application was first inspected against Nielsen’s ten heuristics (Galavi et al., 2024). The inspection used the screens in Figure 3 and a cognitive walkthrough of the “check a photo, then read recommendations” task. Severity was rated from 0 (not a problem) to 4 (catastrophe). Because the evaluator also built the system, problems may be missed or rated leniently, so these findings are complemented by the participant survey below.

Table 15: Functional test summary (details in Appendix C)

Area	Cases	Pass	Observation
Analysis, valid input	5	5	Skin type correctly changes ranking (TC18/19); a false face rejection occurred in the first run (TC02)
Health & input validation	6	6	Correct 400/422 codes; the corrupt-image message exposes an internal Python object name
Products & metadata	6	6	Search, filter, 404 and categories correct
Chat	2	1	Empty message rejected; question answered in 55 s in the first run but timed out in the re-run (TC15)
Total	19	18	

Usability: participant survey

Participants used the app on their own phones and then completed an anonymous Google Form (protocol and questions in Appendix D). Each participant carried out four tasks: check a face photo, read and interpret the result, find and save a recommended product, and ask the assistant a question. For each task they reported whether they completed it and rated its ease from 1 (very difficult) to 5 (very easy). They then completed the ten-item System Usability Scale (SUS), scored from 0 to 100, where about 68 is the usual benchmark for average usability (Vlachogianni and Tselios, 2022). Finally, they answered questions on whether they understood the “uncertain” label, noticed the disclaimer, and would trust or buy the recommended products. Responses were exported from Google Forms as CSV and analysed with a script stored in `evidence/user_study/`.

[TODO: two or three sentences summarising the main themes from the open comments, quoting participants only with their permission.]

7.0 Critical Evaluation

SkinCare AI is a working, end-to-end prototype. A user photographs their face and receives a per-concern cosmetic observation with its uncertainty. They can add their skin type and get a short list of products that are in stock at Nepalese retailers, with the reason each was chosen. They can then ask follow-up questions of an assistant that is not allowed to invent products.

This section judges how well the artefact meets its objectives and requirements, how far its evaluation can be trusted, how it compares with the systems reviewed in Section 2, and what the project process itself taught.

Table 16: Heuristic evaluation findings (author-conducted; severity 0–4)

Heuristic	Finding	Sev.
Match with real world	The evening routine lists “broad-spectrum sunscreen” because the routine is split into morning and evening by position, not by step type	3
Aesthetic & minimalist	Chat replies show raw Markdown (bold) instead of formatted text	2
Consistency & standards	Settings chips truncate labels (“Not provided” is clipped; “Combination” and “Sensitive” overlap) on a 390 px screen	2
Consistency & standards	The About panel lists “Jeevee · Oriflame” but omits ForEveryNG, which supplies 41% of products	1
Error prevention	Close-up photos are rejected with “No face found” although the concern model can analyse them	3
Visibility of system status	Chat gives no progress indication during a response that can take about 55 s on CPU	2
Help users with errors	Rejections give an actionable retake instruction; network errors are translated into plain language	0
Recognition vs. recall	Score bars show a threshold tick and a status label, so users need not remember what a score means	0
Accuracy of content	Some products sit in the wrong category (a body wash appears under “toner”) because of keyword categorisation	2
Code quality (dev)	React warning: <button> nested in <button> on the history and result screens (invalid markup; accessibility risk)	1

Achievement of objectives

Requirements check

Table 19 checks every requirement in Table 4 against the test evidence. All requirements are present; four are only partly met because testing exposed a specific weakness.

Model results

The central technical result is mixed, and it should be stated plainly. The deployed model is *useful for fine lines, moderate for blemishes and redness, and not yet useful for dark spots or visible pores*. The interim target of a weighted F1 of at least 80% was, in effect, replaced. A macro F1 of 0.789 could be presented as “close to target”. That would be misleading, because F1 rewards always predicting “present” when positives far outnumber negatives (Chicco and Jurman, 2020). Reporting accuracy, balanced accuracy and ROC-AUC next to F1 is less flattering but truthful.

The sequence of model versions (Table 13) is the clearest evidence in the project about what limits performance:

Table 17: Participant survey results (n = [TODO])

Measure	Result
Participants	[TODO: n, age bands, skin types]
Task 1: check a photo	[TODO: completion %, mean ease]
Task 2: read the result	[TODO]
Task 3: find and save a product	[TODO]
Task 4: ask the assistant	[TODO]
SUS score	[TODO: mean (SD), range]
Understood “uncertain”	[TODO: % yes]
Noticed disclaimer	[TODO: % yes]
Would trust / buy recommendations	[TODO: mean 1–5]

- **Changing the loss did not help.** Class reweighting targets imbalance directly, yet it lowered test F1 and reduced redness specificity to zero.
- **Adding more positives did not help.** Extra melasma photos raised ROC-AUC slightly but lowered accuracy, and dark spots stayed near chance.
- **Adding reviewed negatives did help.** Filling unknown labels in existing training images raised the number of negative training labels from 9 to 159 for blemishes and from 19 to 188 for redness. This produced the only candidate that improved all four macro metrics, with the largest gains on exactly those concerns.

Together with the learning curve (Figure 5), which was still rising for most concerns, these results show that the bottleneck is the quantity and variety of labelled data, especially negatives, and not the network.

The improvement must also be kept in proportion. It is a single stored training run, and same-seed retraining varied by several points on the Apple GPU. The averaged runs still favour the human-reviewed recipe, but the margin over `concern_v2` is modest. The deployed model is also worse than `concern_v2` on dark spots, so the replacement was a trade-off, made because it was better overall and on the concerns with the largest gains.

Validity of the evaluation

Four threats to validity must be acknowledged.

- **The test set is no longer untouched.** Its results were inspected while comparing four model versions. No threshold was ever set on test data, but choosing the deployed model by comparing test results is a form of selection. The set should now be treated as a regression benchmark, and a fresh independent test set is needed for final claims about generalisation.
- **Very small negative counts.** The test set has only 4 negative blemish images and 8 negative redness images. One image more or less changes specificity by 12 to 25 points, so per-concern balanced accuracy for these concerns is unstable.

Table 18: Objective attainment

Objective	Status	Evidence and judgement
O1 Dataset	Met	26,000+ images audited; 1,400+ reviewed, plus a second review of 200 training images; 2,000+ images in leakage-free splits. Test negatives remain scarce for three concerns.
O2 Model	Partly met	Four versions trained and compared on one fixed test set; the best (pooled accuracy 76.3%, macro F1 0.789) is deployed. Fine lines are strong (ROC-AUC 0.920); blemishes and redness moderate; dark spots and pores near chance.
O3 Catalogue	Met	1,500+ products from 3 retailers, above the target of 1,000. Ingredient tags cover only 35% of products.
O4 Recommender	Met	Transparent weighted ranking that responds to skin type (TC18/19) and shows matched ingredients. Its suitability has not been rated by experts.
O5 Application	Met	All F1–F10 requirements implemented, four only partly met (Table 19); 18 of 19 functional tests passed.
O6 Evaluation	Met	Model metrics, learning curve, 19 functional tests, heuristic review and a participant SUS survey. No fresh, never-inspected test set.

- **The test population differs from the app’s inputs.** The face gate rejects 22% of test images (Table 14), mostly close-ups. The metrics therefore describe a broader input range than the app accepts.
- **Representation is unmeasured.** No image carries a skin-tone label, so it is unknown how the model performs on Nepalese users. Given Groh et al. (2021), Daneshjou et al. (2021) and Daneshjou et al. (2022), a performance gap for darker skin should be assumed until measured.

Recommender quality

The recommender behaves as designed, but its inputs are thin. Only 35% of products have an inferred target ingredient, so for most candidates the 0.40 ingredient weight contributes nothing. The ranking then falls back on rating and review count, which rewards popular products rather than suitable ones. Keyword categorisation also misplaces some products; for example, a body wash appears in the toner list.

On the positive side, every recommendation can be explained line by line, and stock status and price come from real retailers. Changing the stated skin type visibly changes the ranking: ceramide moisturisers rise for dry skin and oil-free ones for oily skin (TC18/19). No dermatologist has yet judged whether the recommendations are *appropriate*, so clinical suitability remains unvalidated.

Table 19: Requirements against evidence

Req.	Status	Evidence
F1 Upload	Met	TC05, TC07: wrong type and oversize files rejected with clear messages
F2 Face gate	Partly	All 300 non-face images rejected (TC04), but 22% of held-out close-ups also rejected (Table 14)
F3 Per-concern status	Met	TC02, TC03: score, threshold and status for all five concerns
F4 Skin type	Met	TC17–TC19: invalid type rejected; valid type changes the ranking
F5 Recommendations	Met	Up to five products per category with matched ingredients
F6 Routine	Partly	Routine shown, but sunscreen can appear in the evening (Table 16)
F7 Catalogue	Met	TC09–TC13: search, filter, sort, detail and 404
F8 History	Met	On-device history under the Saved tab (Figure 3)
F9 Chat	Partly	Grounded replies in the first run, but no timeout: queued requests blocked the re-run (TC15)
F10 Disclaimer	Met	Shown on result and chat screens
N1 No medical labels	Met	Only cosmetic concerns; red-flag questions are signposted to a dermatologist
N2 Validation-only thresholds	Met	Thresholds stored with their validation source in <code>evaluation.json</code>
N3 No photo storage	Met	Server is stateless; history kept on the device
N4 Analysis under 2 s	Met	Warm analysis well under 1 s (Section 6.2)
N5 Free software only	Met	No paid APIs; Qwen2.5 runs locally
N6 Structured errors	Partly	Correct codes, but the corrupt-image message exposes an internal object name (TC06)

Usability

The heuristic review found two severity-3 problems: sunscreen listed in the evening routine, and close-up photos rejected with “No face found”. Both were confirmed by other evidence (the routine logic and the face-gate test), so they are real defects rather than evaluator opinion. The remaining issues were cosmetic or minor.

The participant survey (Section 6.2, Table 17) adds the user’s view. [TODO: after the responses are analysed, state the mean SUS score against the benchmark of 68 (Vlachogianni and Tselios, 2022), the task with the lowest completion or ease rating, and whether participants understood the “uncertain” label and the disclaimer. Note whether the survey confirmed the heuristic findings.]

Comparison with existing systems

The artefact differs from the systems reviewed in Section 2 in four ways.

1. **Scope honesty.** Clinical systems such as Liu et al. (2020), and most of those reviewed by Jones et al. (2022), target diagnosis using curated clinical data. This project deliberately removed diagnosis after finding that its data could not support it. It presents outputs as observations with an *uncertain* band, so the system fails visibly rather than confidently.
2. **Leakage-aware data engineering.** Many applied skin projects built on Kaggle data report accuracy without controlling for duplicates. This project grouped duplicates across splits, recorded provenance for every source, kept every model version, and published negative results. Its numbers are lower, but they mean what they claim (Kapoor and Narayanan, 2023).
3. **Local market grounding.** Unlike the global recommenders reviewed by Ramrakhiani and Kalbande (2024), every product suggested is sold by a Nepalese retailer, priced in rupees and marked with its stock status.
4. **Code-enforced language-model grounding.** The assistant can only mention catalogue products. This is enforced in code rather than by the prompt alone, which directly limits the hallucination risk described by Ji et al. (2023).

Compared with the interim design, Grad-CAM heatmaps and TF-IDF similarity were not implemented. The explainability gap identified in the interim report is therefore only partly closed: the system offers transparent scores and reasons, but no visual attribution.

Reflection on the project process

The most valuable decision was to stop tuning the first model and audit the data instead. It cost several weeks and forced a change of scope, but without it the project would have reported high accuracy on leaked, mislabelled data. The iterative process made this change possible. Its cost was time: the second label review, the experiments and the user survey all happened late, leaving no time to collect a new, independent test set. With hindsight, the data audit should have been the first task of the project rather than a response to poor results, and label review should have targeted negative examples from the start.

7.1 Impact

For Nepalese consumers, the prototype shows that a free, phone-based first step is feasible. It gives cosmetic guidance tied to purchasable products and a clear signpost to a dermatologist. Its current accuracy does *not* justify public release. Its greatest potential impact is as a validated pipeline into which better data can be fed.

For Nepalese e-commerce, the unified catalogue and recommender could help retailers such as Jeevee or ForEveryNG guide customers to in-stock products by need rather than by brand. The catalogue audit also shows that most listings omit ingredient information. Retailers could fix this cheaply, and doing so would improve both search and recommendation.

For academic practice, the project provides a documented, reproducible example of how aggregated public skin data can mislead. More than half of the images were near-duplicates, and 2,400+ duplicate groups spanned sources. It also shows that targeted human review of negatives improved the model more than changing the loss or adding positives. This supports calls for transparent documentation of datasets and their limitations (Gebru et al., 2021).

Ethically, the pivot away from disease labels reduces the risk of false reassurance. With the earlier design, a user with a serious lesion might have trusted a “no carcinoma” result. The residual risk is that users over-trust cosmetic outputs. This is mitigated, but not eliminated, by uncertainty display and disclaimers.

8.0 Conclusion

This project set out to give Nepalese users trustworthy skincare guidance from a single photograph, linked to products they can actually buy. It delivered a complete mobile prototype. A YuNet face gate, a multi-label EfficientNetB0 concern model, a catalogue of 1,500+ Nepalese products, a transparent weighted recommender and a grounded Qwen2.5 assistant are integrated behind a FastAPI backend and an Expo client.

The most important moment in the project was not a modelling choice but a data audit. The audit revealed that the original data was dominated by duplicates, stock imagery and unverifiable labels. On that evidence the scope was narrowed: from diseases and photo-based skin type to five human-labelled cosmetic concerns. This made the system safer and its results meaningful.

Those results are modest but improving. Four model versions were compared on one fixed held-out set. The deployed model, retrained after a second review that added many negative labels, reached a pooled accuracy of 76.3%, a macro F1 of 0.789 and a macro ROC-AUC of 0.734, the best of the four. Fine lines are detected reliably (ROC-AUC 0.920), blemishes and redness moderately, and dark spots and visible pores barely better than chance. Reweighting the loss and adding melasma photos both failed, while adding reviewed negatives helped, which confirms that the limiting factor is the data. Eighteen of 19 functional tests passed; the chat assistant timed out when requests queued. Testing also exposed a mismatch between the face gate and the model, and several interface defects that are now documented.

The project therefore succeeds as an engineering artefact and as an honest baseline. It does not yet succeed as a consumer product. Its value lies in a working pipeline from photo to purchasable product, a reproducible leakage-aware protocol, and a clear, evidence-based account of what must improve next.

8.1 Limitations

Too few test negatives and no skin-tone labels.

The test set has only 4, 8 and 9 negative images for blemishes, redness and pores. A single image changes their specificity by 11 to 25 points. Nothing is known about performance by skin tone. *Impact*: claims beyond fine lines are weak, and fairness is unmeasured. *Addressed by*: reporting balanced accuracy and ROC-AUC next to F1, and stating the gap explicitly.

Test set reused for model choice.

The same test set was inspected while comparing four model versions, and the deployed model was chosen partly on those results. *Impact:* the reported metrics are a regression benchmark, slightly optimistic, and not an independent estimate. *Addressed by:* never setting thresholds on test data, and flagging the issue.

Training not fully reproducible.

On the Apple GPU, the same recipe and seed gave 72.2% and 78.9% macro accuracy. *Impact:* differences of a few points between single runs are within noise, so the gain over `concern_v2` is modest. *Addressed by:* storing each deployed model's weight hash and reporting averaged runs.

Dark-spot regression.

The deployed model is worse than `concern_v2` on dark spots (accuracy 50.6% versus 64.4%), and no version separates dark spots well. *Impact:* dark-spot output is unreliable and should not drive recommendations on its own.

Labels from one reviewer, and not new data.

All human labels come from a single reviewer, and inter-rater agreement was not measured. The second review re-labelled existing training images rather than adding new people. *Impact:* label noise is unknown, and the data's variety did not grow.

Gate-model mismatch.

22% of test images, mostly close-ups, would be rejected by the app. *Impact:* users who photograph close-ups are turned away, and the metrics cover inputs the app does not accept.

Sparse catalogue metadata.

65% of products have no inferable target ingredient, and 18% have no category. *Impact:* the ranking leans on popularity, and some products appear in the wrong category.

Small, self-selected survey.

Survey participants were a convenience sample recruited by the author, used the app briefly and unsupervised, and rated perceived usability only. *Impact:* the results show first impressions, not long-term use, and cannot judge whether the observations or recommendations are correct.

Chat latency and prototype hosting.

CPU-based language-model responses take about a minute, requests are processed one at a time with no timeout, and the system runs only on a local network.

8.2 Future Work

1. **Collect targeted negatives and a fresh test set.** This is the highest priority. With ethics approval and consent, collect full-face photos from Nepalese volunteers across Fitzpatrick types, deliberately including people *without* each concern. Hold out a new test set that is never inspected during development, and report metrics by skin tone (Groh et al., 2021).

2. **Measure label reliability.** Have a second annotator, ideally with cosmetology or dermatology training, label a subset, and report Cohen's κ for each concern. Revise the definition for dark spots, where every model version stayed near chance.
3. **Make training reproducible.** Train final models with deterministic settings (or on CPU) and report the mean and spread over several seeds rather than a single run.
4. **Align the gate and the model.** Either train and evaluate only on full-face images, or accept close-ups with a separate quality check. Report metrics on exactly the images the app accepts.
5. **Fix the defects found in testing.** Assign routine steps to morning or evening by step type, render Markdown in chat, fix the chip layout and nested buttons, sanitise the corrupt-image message, and add a progress indicator and a timeout to chat, so an abandoned request cannot block later ones.
6. **Enrich the catalogue.** Scrape full INCI ingredient lists where available, validate categories with a small classifier, and test whether embedding similarity then beats the weighted rules using expert-rated relevance (for example, `precision@5`).
7. **Extend the user study.** Repeat the survey with a larger and more varied sample, including older users and people outside Kathmandu, add observed (moderated) sessions, and iterate on the findings.
8. **Add visual explanation and deployment.** Add Grad-CAM overlays (Selvaraju et al., 2020) for concerns marked *present*. Export the model to TensorFlow Lite for on-device inference, which also improves privacy, and add Nepali-language support.

References

- Barz, B. and Denzler, J. (2020), 'Do we train on test data? Purging CIFAR of near-duplicates', *Journal of Imaging* **6**(6), 41.
- Chicco, D. and Jurman, G. (2020), 'The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation', *BMC Genomics* **21**(1), 6.
- Chicco, D., Tötsch, N. and Jurman, G. (2021), 'The Matthews correlation coefficient (MCC) is more reliable than balanced accuracy, bookmaker informedness, and markedness in two-class confusion matrix evaluation', *BioData Mining* **14**(1), 13.
- Daneshjou, R., Smith, M. P., Sun, M. D., Rotemberg, V. and Zou, J. (2021), 'Lack of transparency and potential bias in artificial intelligence data sets and algorithms: A scoping review', *JAMA Dermatology* **157**(11), 1362–1369.
- Daneshjou, R., Vodrahalli, K., Novoa, R. A., Jenkins, M., Liang, W., Rotemberg, V. et al. (2022), 'Disparities in dermatology AI performance on a diverse, curated clinical image set', *Science Advances* **8**(32), eabq6147.
- Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. and Zisserman, A. (2010), 'The PASCAL visual object classes (VOC) challenge', *International Journal of Computer Vision* **88**(2), 303–338.
- Galavi, Z., Norouzi, S. and Khajouei, R. (2024), 'Heuristics used for evaluating the usability of mobile health applications: A systematic literature review', *Digital Health* **10**, 20552076241253539.
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H. and Crawford, K. (2021), 'Datasheets for datasets', *Communications of the ACM* **64**(12), 86–92.
- Goldust, M. and Grant-Kels, J. M. (2024), 'Using AI to help address skin health challenges caused by climate change', *International Journal of Dermatology* **63**(7), e126–e127.
- Groh, M., Harris, C., Soenksen, L., Lau, F., Han, R., Kim, A., Koochek, A. and Badri, O. (2021), Evaluating deep neural networks trained on clinical images in dermatology with the Fitzpatrick 17k dataset, in 'Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops', pp. 1820–1828.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A. and Fung, P. (2023), 'Survey of hallucination in natural language generation', *ACM Computing Surveys* **55**(12), 1–38.
- Jones, O. T., Matin, R. N., van der Schaar, M., Prathivadi Bhayankaram, K., Ranmuthu, C. K. I., Islam, M. S. et al. (2022), 'Artificial intelligence and machine learning algorithms for early detection of skin cancer in community and primary care settings: A systematic review', *The Lancet Digital Health* **4**(6), e466–e476.

- Kapoor, S. and Narayanan, A. (2023), 'Leakage and the reproducibility crisis in machine-learning-based science', *Patterns* **4**(9), 100804.
- Karras, T., Laine, S. and Aila, T. (2021), 'A style-based generator architecture for generative adversarial networks', *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**(12), 4217–4228.
- Liu, W., Wang, H., Shen, X. and Tsang, I. W. (2022), 'The emerging trends of multi-label learning', *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(11), 7955–7974.
- Liu, Y., Jain, A., Eng, C., Way, D. H., Lee, K., Bui, P., Kanada, K., de Oliveira Marinho, G., Gallegos, J., Gabriele, S. et al. (2020), 'A deep learning system for differential diagnosis of skin diseases', *Nature Medicine* **26**(6), 900–908.
- Moon, J., Chung, H. and Jang, I. (2024), Facial wrinkle segmentation for cosmetic dermatology: Pretraining with texture map-based weak supervision, in 'Proceedings of the 27th International Conference on Pattern Recognition (ICPR 2024)', Springer, Cham. doi:10.1007/978-3-031-78104-9_22.
- Paudel, V. (2020), 'The increasing scope of teledermatology in Nepal', *Journal of Nepal Medical Association* **58**(232), 1024–1026.
- Qwen Team (2024), 'Qwen2.5 technical report', arXiv preprint arXiv:2412.15115.
- Ramrakhiani, N. and Kalbande, D. (2024), 'A comprehensive review of AI-powered skincare product recommendation systems: From data collection to user experience', *Digital Health* **10**, 1–18.
- Ricci, F., Rokach, L. and Shapira, B., eds (2022), *Recommender Systems Handbook*, 3rd edn, Springer, New York. doi:10.1007/978-1-0716-2197-4.
- Roy, D. and Dutta, M. (2022), 'A systematic review and research perspective on recommender systems', *Journal of Big Data* **9**(1), 59.
- Schröer, C., Kruse, F. and Gómez, J. M. (2021), 'A systematic literature review on applying CRISP-DM process model', *Procedia Computer Science* **181**, 526–534.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2020), 'Grad-CAM: Visual explanations from deep networks via gradient-based localization', *International Journal of Computer Vision* **128**(2), 336–359.
- Tan, M. and Le, Q. (2019), EfficientNet: Rethinking model scaling for convolutional neural networks, in 'Proceedings of the 36th International Conference on Machine Learning', pp. 6105–6114.
- Vlachogianni, P. and Tselios, N. (2022), 'Perceived usability evaluation of educational technology using the System Usability Scale (SUS): A systematic review', *Journal of Research on Technology in Education* **54**(3), 392–409.

- Ward, A., Li, J., Wang, J., Lakshminarasimhan, S., Carrick, A., Campana, B., Hartford, J., Sreenivasaiah, P. K., Tiyasirisokchai, T., Virmani, S. et al. (2024), 'Creating an empirical dermatology dataset through crowdsourcing with web search advertisements', *JAMA Network Open* **7**(11), e2446615.
- Wu, W., Peng, H. and Yu, S. (2023), 'YuNet: A tiny millisecond-level face detector', *Machine Intelligence Research* **20**(5), 656–665.
- Wu, X., Wen, N., Liang, J., Lai, Y.-K., She, D., Cheng, M.-M. and Yang, J. (2019), Joint acne image grading and counting via label distribution learning, *in* 'Proceedings of the IEEE/CVF International Conference on Computer Vision', pp. 10642–10651.
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H. and He, Q. (2021), 'A comprehensive survey on transfer learning', *Proceedings of the IEEE* **109**(1), 43–76.

Appendix A: Project Timeline

Figure 6 shows the actual schedule. Dates from August 2026 onwards are taken from the Git history. Earlier dates follow the interim report's plan.

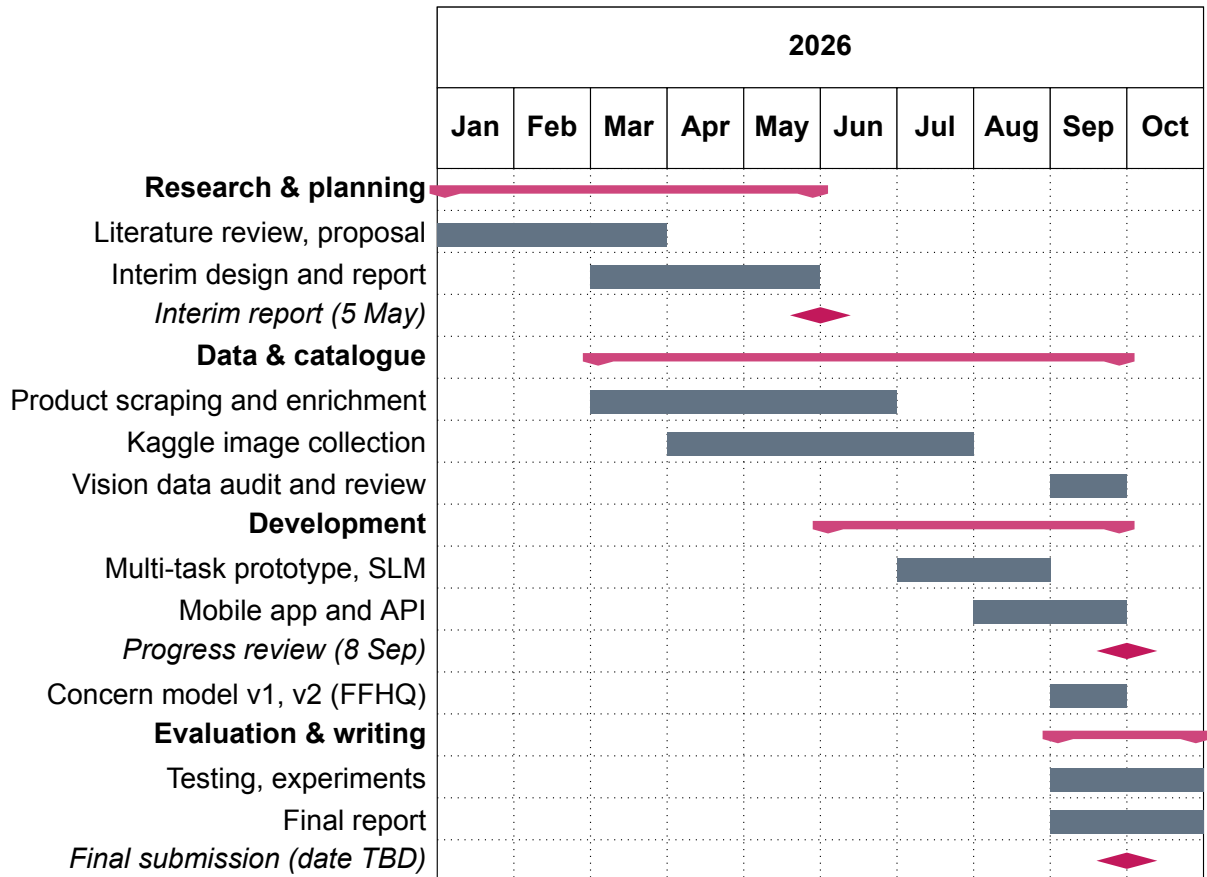


Figure 6: Project Gantt chart

Appendix B: Feasibility, Risks, and Ethical Aspects

B.1 Revised Feasibility

The project proved feasible within its revised scope.

- All software is free and open source.
- Training fits on a laptop, because EfficientNetB0 with 1,200+ images trains in minutes per phase on an Apple M-series CPU.
- Analysis runs in under 0.25 s once the model is loaded.

The binding constraints were the following:

- *Labelled data*: human review is slow, and public cosmetic datasets are small.

- *Time*: the September pivot left little time for the participant study and no time to collect a new test set.
- *Compute for the language model*: CPU chat responses take about a minute.

The core flow does not depend on the language model, so this last constraint did not threaten delivery.

B.2 Updated Risk Analysis and Mitigation

Table 20: Risk assessment and mitigation (updated with outcomes)

Potential risk	Likelihood	Impact	Mitigation strategy and outcome
Small or contaminated dataset causing misleading accuracy	High	High	Audited duplicates, split by duplicate group, human review. <i>Occurred</i> ; mitigated by the scope change and honest metrics.
Poor performance on Nepalese skin tones	High	High	Report as unmeasured; plan skin-tone-stratified collection. <i>Open</i> .
Language model hallucinating products or ingredients	Medium	High	Candidate-only prompt, code-level ID validation, rule fallback. <i>Mitigated</i> (guardrail test passes).
Users treating output as diagnosis	Medium	High	No disease labels, “uncertain” band, persistent disclaimer. <i>Reduced</i> .
Scraper blocked or site layout changes	Medium	Medium	Snapshot CSVs kept; three independent sources. <i>Did not occur</i> .
Language-model latency too high	High	Low	Made optional; rule fallback. <i>Occurred</i> (about 55 s); core flow unaffected.
Insufficient time for user study	High	Medium	Short online survey (Google Forms) alongside a heuristic review. <i>Mitigated</i> ; sample is small.

B.3 Revised Ethical Considerations

Human participants. The usability survey followed the protocol in Appendix D. Participants were adults who gave informed consent on the first page of the form. The form collected no names, emails or photos, and participants could use a sample photo instead of their own. Any photo used in the app is processed in memory and never stored. [TODO: state the ethics approval or supervisor sign-off obtained before the survey was sent.]

Third-party data. Every image source was recorded with its licence. Sources with unclear provenance were rejected or limited to partial use. FFHQ-Wrinkle (CC BY-NC-SA) and ACNE04 (academic use) are used non-commercially with attribution. SCIN images were downloaded under the SCIN Data Use Licence, which prohibits re-identification. Their privacy masks were

left untouched, and the images were quarantined unused. Raw images are not committed to the repository.

Scraping. Only publicly listed product information was collected, at a low request rate, for non-commercial academic use. No customer data was collected.

Privacy in the app. Uploaded photos are processed in memory and are not stored on the server. Scan history and favourites stay on the user's device.

Harm and fairness. Removing disease labels avoids false medical reassurance. Every result states that it is a cosmetic observation and not a diagnosis. The known risk of lower accuracy on under-represented skin tones is stated openly rather than hidden (Daneshjou et al., 2021; Groh et al., 2021). A public release should wait until performance has been measured on the intended population.

Academic integrity. All reported numbers come from stored evaluation files or recorded test outputs. Heuristic findings are labelled as author-conducted, and survey results are taken only from the exported Google Forms responses, analysed with the script in `evidence/user_study/`.

Appendix C: Functional Test Cases

These cases were first executed on 30 September 2026 and re-run on 1 October 2026 after the human-reviewed model was deployed. The table shows the re-run (raw output in `evidence/`; the first run is kept alongside it with the suffix `_20260930`).

ID	Input	Expected	Actual	Result
TC01	GET /api/health	200, status ok	200, status ok	Pass
TC02	ACNE04 face image, skin type oily	200, concerns and recommendations	200; 3 present, 2 absent; 6 categories × 5	Pass [†]
TC03	FFHQ face, no skin type	200, 5 concerns, recommendations	200; fine lines present, 4 absent; 4 categories × 5	Pass
TC04	PASCAL VOC boat photo	200, rejected, retake message	200, rejected, retake message	Pass
TC05	Text file (text/plain)	400 "File must be an image"	400 as expected	Pass
TC06	Corrupt bytes labelled image/jpeg	422	422, message exposes Python object name	Pass*
TC07	11 MB file	400 "Image too large"	400 as expected	Pass
TC08	Missing image field	422	422 field required	Pass
TC09	List products, page size 5	200, 5 products	200, 5 products	Pass

ID	Input	Expected	Actual	Result
TC10	Filter sunscreen + oily	200, matching products	200, SPF gel products	Pass
TC11	Search “niacinamide”	200, relevant products	200, niacinamide products	Pass
TC12	Unknown product ID	404	404 “Product not found”	Pass
TC13	GET categories	200, category list	200, 11 categories	Pass
TC14	GET skin types	200, 5 skin types	200, with ingredients	Pass
TC15	Chat “What helps with dark spots?”	200, grounded reply	No response within 240 s (first run: grounded reply in 54.9 s)	Fail
TC16	Chat with empty message	422	422 string too short	Pass
TC17	Invalid skin type “purple”	422	422 “Unknown skin type”	Pass
TC18	FFHQ face, skin type dry	Ranking favours dry-skin products	Ceramide moisturisers ranked top	Pass
TC19	Same face, skin type oily	Ranking favours oily-skin products	Oil-free moisturisers ranked top	Pass

*Passed with an observation recorded in Table 15. † In the first run a different ACNE04 image was wrongly rejected by the face gate; the gate is unchanged, so such false rejections still occur (Table 14).

Appendix D: Annotation Guide and Usability Study Protocol

D.1 Annotation guide (summary)

Reviewers mark only what is visibly supported by the image. *Unknown* is used whenever framing, lighting, resolution, makeup or occlusion prevents a judgement. Uncertainty is never converted to *absent*. An image is marked unusable if any of the following applies:

- it is not facial skin;
- it is a collage or heavily edited;
- it has a dominant watermark;
- it is badly exposed.

Label	Present when	Not sufficient alone
Blemishes	Visible pimples, inflamed spots, blackheads or whiteheads	Scars or pigmentation without active blemishes
Dark spots	Localised darker marks or visibly uneven pigmentation	Shadows, freckles alone, lighting gradients
Redness	Redness distinguishable from normal tone	Warm lighting, blush, colour cast
Visible pores	Clearly visible pores at usable resolution	Compression noise, extreme zoom texture
Fine lines	Clearly visible fine lines or wrinkles	Folds caused by a single exaggerated expression

D.2 Usability survey protocol

Participants: adults living in Nepal, recruited by convenience sampling from the author’s contacts, with varied skin types. Participation was voluntary and anonymous.

Procedure (about 20 minutes per participant):

1. Participant opens the Google Form and gives consent on its first page.
2. Participant opens SkinCare AI (Expo build on the same network as the backend) and optionally sets a skin type.
3. Task 1: check one face photo. A sample photo is offered, so using their own image is optional.
4. Task 2: open the result and read what each concern means.
5. Task 3: find a recommended product and save it to the shelf.
6. Task 4: ask the assistant one question about the result.
7. Participant completes the rest of the form: task completion and ease, the ten SUS items, understanding and trust questions, and open comments.

Form sections (created by `evidence/user_study/create_form.gs`):

1. Information and consent (required).
2. About you: age group, self-reported skin type, prior use of skincare apps, own or sample photo.
3. Tasks 1–4: completed (yes without help / yes with help / no) and ease (1–5).
4. System Usability Scale, ten items (Vlachogianni and Tselios, 2022).
5. Understanding and trust: meaning of “uncertain” (multiple choice), disclaimer noticed, results match what is visible, trust in recommendations, usefulness of ingredient reasons, intention to buy (1–5 each).
6. Open comments: what was liked, what was confusing.

Analysis: the responses CSV is processed by `analyse_sus.py`, which computes each participant's SUS score (odd items $r - 1$, even items $5 - r$, sum $\times 2.5$), the mean and standard deviation, task completion rates, mean ease ratings and answer counts.

Consent statement: "Taking part is voluntary and you may stop at any time without giving a reason. Any photo you use in the app is analysed in memory and is not stored. Your answers are anonymous and will be used only in the project report. The app gives cosmetic observations, not medical advice."

SUS items (rated 1 = strongly disagree to 5 = strongly agree; the form uses "app" in place of "system"):

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

Appendix E: Key Code Excerpts

Weighted masked multi-label loss (`src/model/concern_model.py`). Unknown labels (-1) are excluded from the loss:

```
def weighted_masked_bce(labels, predictions, weights):  
    """Weighted BCE averaged over known labels; -1 labels are ignored."""  
    labels = tf.cast(labels, tf.float32)  
    weights = tf.cast(weights, tf.float32) * tf.cast(labels >= 0, tf.float32)  
    safe_labels = tf.maximum(labels, 0.0)  
    loss = tf.keras.backend.binary_crossentropy(safe_labels, predictions)  
    return tf.math.divide_no_nan(tf.reduce_sum(loss * weights), tf.reduce_sum(weights))
```

Uncertainty band (`src/inference/predict.py`):

```
if abs(score - threshold) <= self.uncertainty_margin: # 0.05  
    status = "uncertain"  
else:  
    status = "present" if score > threshold else "absent"
```

Language-model guardrail (`src/slm/recommender.py`). Only candidate products survive:

```
allowed = {str(c["product_id"]): c for c in candidates}  
for item in data.get("chosen", []):max_total]:  
    candidate = allowed.get(str(item.get("product_id")))  
    if candidate is None or candidate["product_id"] in seen:  
        continue
```